TECHNICAL REPORT · CAPACITY PLANNING

Probabilistic CPU Capacity Estimation from Transaction Load

Alexander Apartsin

Amdocs · 2011

Capacity-planning study · Transaction-processing system, application and database tiers

ABSTRACT

Sizing hardware capacity for a complex, multi-tier transaction-processing system is a costly trade-off: over-provisioning wastes infrastructure budget, while under-provisioning drops billing transactions and exposes the operator to contractual fines for missed service levels. Sound sizing therefore requires weighing hardware cost estimates against the expected penalty cost of overload, which in turn requires knowing the probability of overload at a given load. This report presents a probabilistic method for that question: given a transaction rate, what is the probability that CPU utilization stays below the level required to meet service-level objectives? Transaction arrivals were modeled as a Poisson process, and CPU utilization conditional on load with a log-normal distribution derived from a multiplicative-overhead argument. Model parameters were learned from one month of peak-hour production measurements. The fitted model reproduced the observed utilization data and yields, for any utilization budget B and candidate transaction rate N_0, an estimate of $P(C < B \mid N = N_0)$. A controlled load experiment at 70%, 100%, and 120% of nominal load supported the modeling choices and revealed a non-CPU bottleneck at extreme load; a wider validation on 26 production servers over 30 days confirmed the log-normal fit inside every hour-of-day bin, and exposed that expected CPU grows exponentially, not linearly, with offered load. The capacity distribution is then composed with an event-arrival histogram and an empirical service-success curve into a decision-support model that returns expected availability and expected revenue under a tiered SLA contract. Applied on two large production deployments, the method supported hardware sizing decisions that avoided provisioning costs on the order of $10 M on the first engagement, and gave a quantitative comparison basis on the second. The method is delivered as an interactive spreadsheet tool for operations teams.

1 Problem statement

Let C denote CPU utilization measured over a 5-second interval, N the average number of transactions per hour, B the maximum CPU utilization at which service-level agreements are still met, and A a tolerance threshold. The capacity-planning task is:

$$\textit{find the maximum } N_0 \textit{ such that } P(C < B \mid N = N_0) > 1 - A$$

In words: the highest sustained transaction rate for which the system stays under its CPU budget with the required confidence. A probabilistic formulation is needed because utilization at a fixed load is not a single number; short-interval measurements fluctuate, and the SLA is violated by the tail of that fluctuation, not by its mean.

2 Load model

The hourly rate N is converted to the 5-second measurement scale as $N_5 = N / (60 \cdot 12)$. The number of transactions n arriving in a given 5-second interval is a random variable, modeled as Poisson:

$$n \sim Poisson(N_5) \approx Normal(\mu = N_5,\ \sigma^2 = N_5)$$

with the normal approximation applying at the arrival counts of interest. Supporting analysis of the production traces shows a near-uniform distribution of transaction volume across days, across weekdays, and of peak-hour transaction counts from day to day, which is consistent with treating the peak-hour arrival process as homogeneous.

3 CPU utilization model

Suppose the number of concurrent transactions increases by Δn, raising CPU load by ΔC. The increment is larger when the system is already busy, because context-switch overhead grows with utilization. The simplest model with this property makes the increment proportional to the current level:

$$\Delta C / \Delta n \propto \alpha C$$

Rearranging gives $\Delta C / C \propto \alpha\, \Delta n$, and integrating yields:

$$\log C = \alpha n + \beta$$

Since n is approximately normal (Section 2), log C is normal, and C follows a standard log-normal distribution. The two parameters α and β are learned from data.

4 Parameter estimation

Parameters were fitted on one month of production measurements from a single host:

- CPU utilization samples collected during the daily peak hour over 30 days (roughly 310 five-second observations);
- daily peak-hour transaction counts over the same 30 days (30 observations).

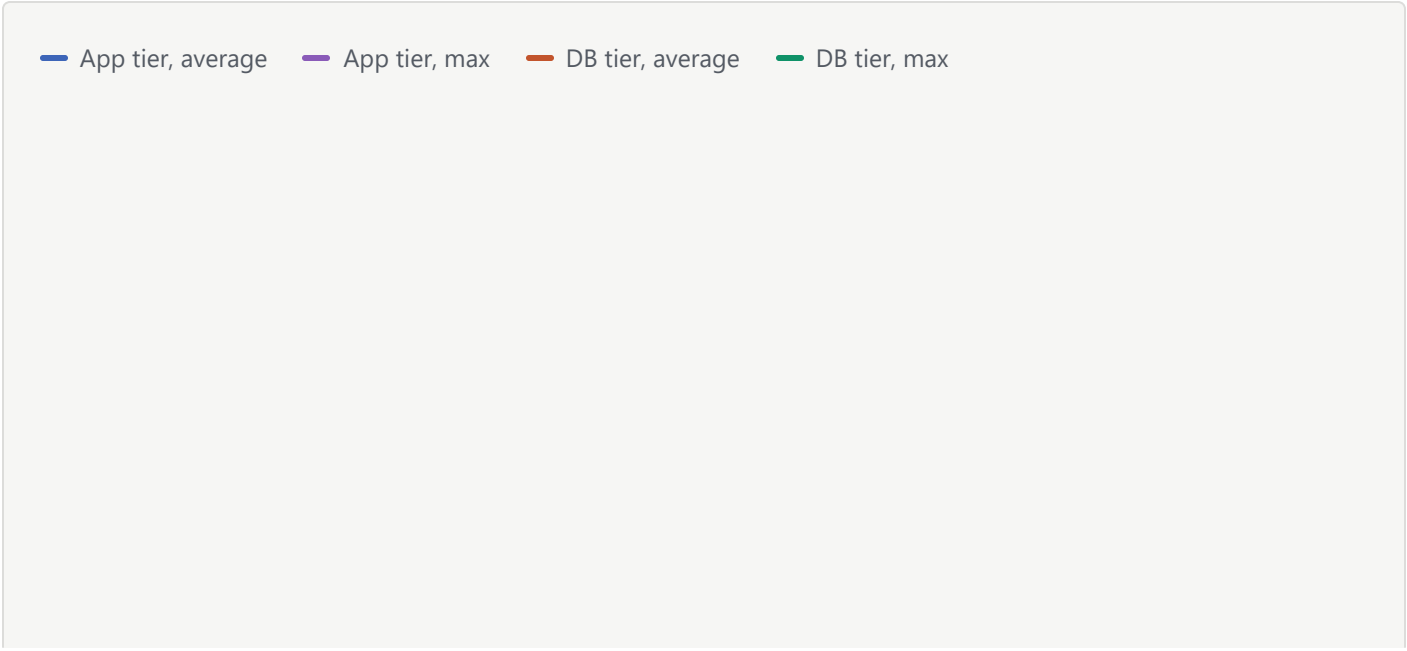
The combined set of about 340 points determines the log-normal parameters by regression of log C on n. The fitted distribution reproduces the observed utilization data well.

5 Controlled load experiment

5.1 Setup

A load experiment on a test environment ran four configurations: 70% of nominal load for 1 hour, 70% for 17 hours (with one missed hour), 100% for 4 hours, and 120% for 1 hour. CPU utilization was recorded on the database tier every 5 minutes (as averages, per the monitoring configuration) and on the application tier as hourly average, minimum, and maximum.

5.2 Observed utilization



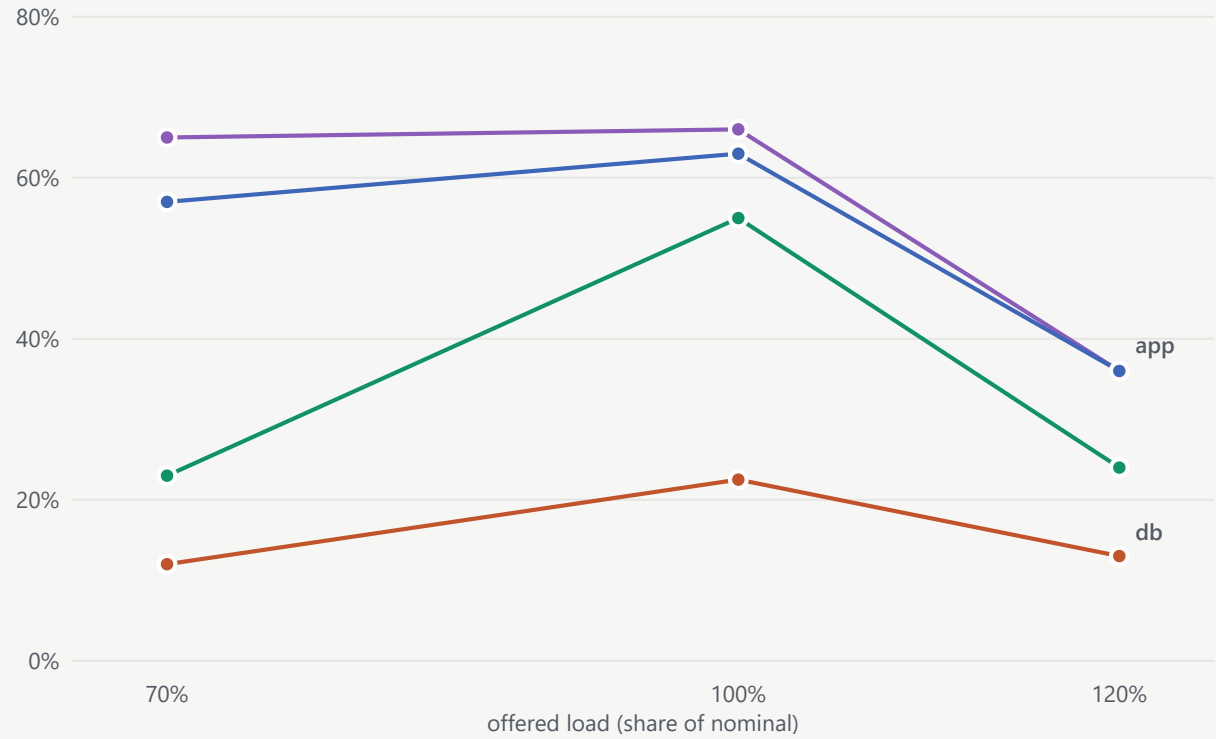


Figure 1. Measured CPU utilization versus offered load. Utilization on both tiers rises from 70% to 100% load, then falls sharply at 120% load: application-tier average drops from 63% to 36% and database-tier max from 55% to 24%. Values read from 5-minute (database) and hourly (application) monitoring aggregates.

The drop at 120% load means CPU stops being the limiting resource at extreme load: throughput is capped elsewhere, with network or disk I/O the leading candidates, and the CPUs idle while work queues upstream. Measurements at 120% load therefore do not describe the CPU-vs-load relationship and are excluded from model fitting.

5.3 Candidate models

The experiment produced too few distinct load levels to validate a complex model directly, so two candidate models constrained by prior experience were compared, both fitted on the 70% and 100% configurations:

- **Model A (normal/linear).** CPU utilization at a fixed load is normally distributed; the mean follows a two-point linear regression on load.
- **Model B (log-normal).** CPU utilization at a fixed load follows the log-normal distribution of Section 3, previously validated on a separate class of production load events.

Model A · normal / linear

Model B · log-normal

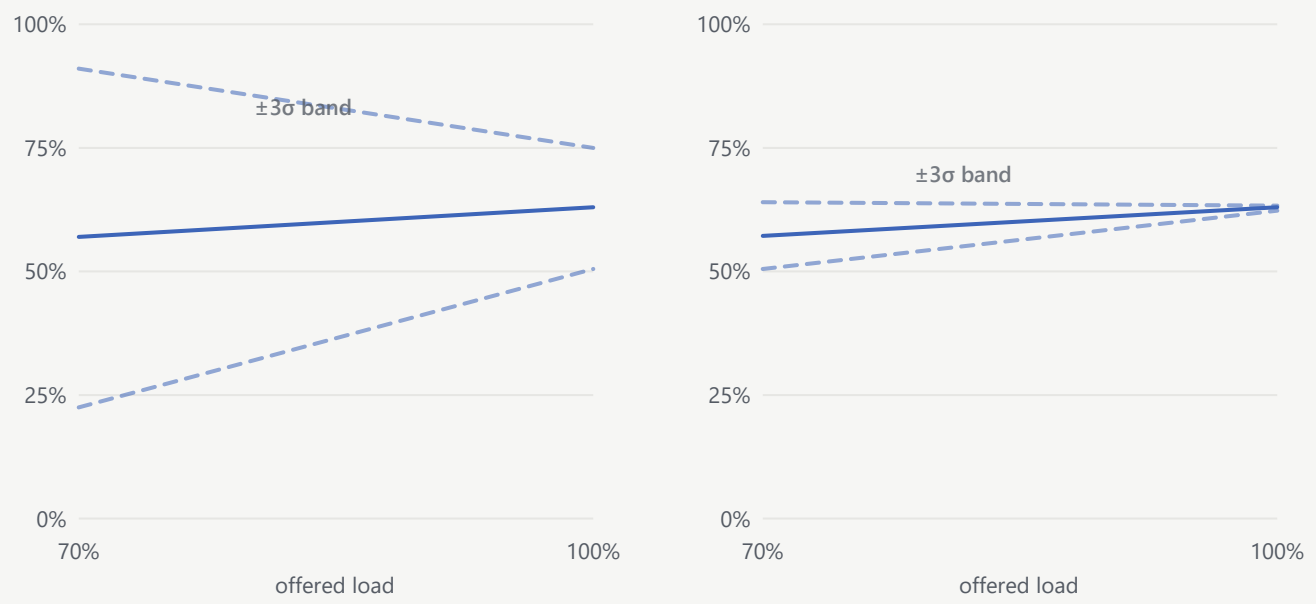


Figure 2. Application-tier mean CPU (solid) with ±3σ envelopes (dashed) under the two candidate models, fitted on the 70% and 100% load configurations and drawn on a common 0 to 100% scale. Model A's normal assumption produces a wide envelope, roughly 22% to 91% at 70% load; Model B's log-normal envelope is far tighter and narrows toward 100% load. The narrower variance estimate at 100% load partly reflects the shorter measurement window (4 hours at 100% versus 17 hours at 70%).

6 Production validation at scale

The controlled experiment supports the modeling choice on a small number of load levels; a wider validation was carried out on a production billing platform. Data were collected from 26 rating and balancing servers over 30 days, spanning four event classes (voice, messaging, data, and replenishment). For every hour-of-day bin, per-server, we retained about 30 event totals (one per day) and about 720 five-second CPU samples, totalling roughly 21,600 CPU measurements per bin. In every bin the empirical distribution of log(CPU) is close to normal, so C follows a log-normal distribution as in Section 3.

Under this fit, an operational capacity ceiling is defined as

$$C_{top} = exp\{\ \mu_{\log C} + 3\ \sigma_{\log C}\ \}$$

with the mean and standard deviation of log C computed inside the bin. By the three-sigma rule this ceiling covers C with 99.8% probability. The exceedance function $P(C > c)$ then follows directly:

Empirical vs. fitted log-normal	Exceedance function $P(C > c)$

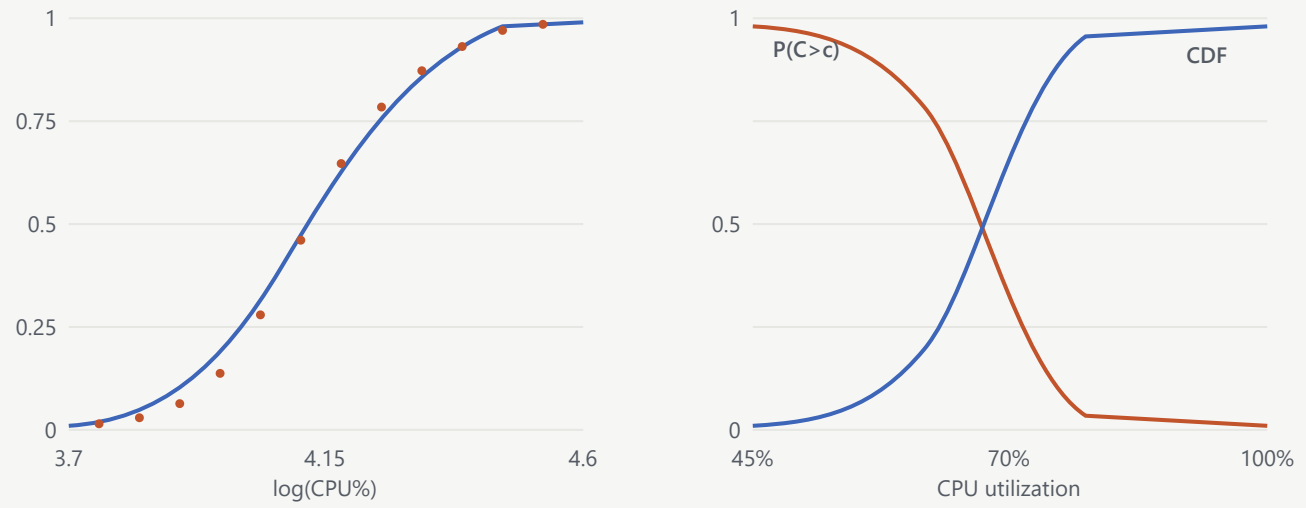


Figure 3. Left: for one representative server-hour, the empirical cumulative frequency of log(CPU%) (points) tracks the fitted normal CDF (line) closely across the range 3.7 to 4.6. Right: the resulting CPU CDF and its complement, the exceedance function $P(C > c)$, used to compute the operational ceiling C_{top}.

7 How the model parameters scale with load

Fitting the log-normal per hour-of-day bin on one heavily-loaded server exposes how $\mu_{\log C}$ and $\sigma_{\log C}$ depend on offered load, expressed as the Poisson intensity λ of transactions per 5-second interval (equivalently, average events per second per core):

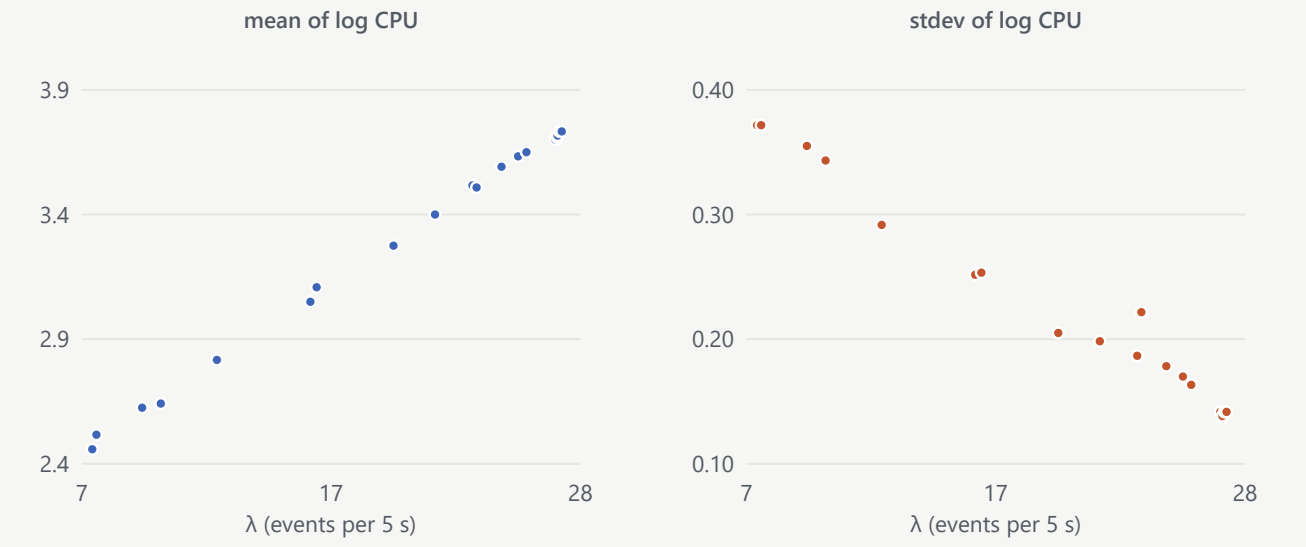


Figure 4. Fitted log-normal parameters versus offered load, one point per hour-of-day bin for a representative server. Mean log CPU rises from about 2.45 at $\lambda = 7$ to about 3.73 at $\lambda = 28$, which corresponds to average CPU utilization growing from roughly 12% to 42%; the standard deviation of log CPU falls from about 0.37 to 0.14. The mean-in-log-CPU rising roughly linearly with λ is consistent with an exponential (not linear) growth of expected CPU with offered load, which matters for long-horizon capacity forecasts.

8 From capacity to economics: decision-support model

The capacity model is a probability distribution over CPU given load; a sizing decision, however, is made against revenue and penalties. To bridge the two we combine three probabilities, each estimated from data or an operating policy:

- $P_e(E = e \mid T = t)$ — probability of e events arriving in the time window at hour t-of-day (event-load histogram);
- $P_c(C = c \mid E = e)$ — probability of CPU utilization c given e events (the log-normal capacity model of Sections 3 and 6);
- $P_s(C = c)$ — probability that an event arriving while CPU is at c is processed within the SLA (a service-success curve).

Summed over the joint distribution, these give the expected number of successful and failed events per server, and thence an expected availability ratio and an expected revenue:

$$S = \Sigma_{c,e,t}\, P_s(C{=}c)\, P_e(E{=}e|T{=}t)\, P_c(C{=}c|E{=}e)$$

$$A = S / (S + F) \;\cdot\; R = RevenueShare(A) \cdot ARPE \cdot S - Fine(A)$$

where ARPE is the operator's average revenue per event and the piecewise functions RevenueShare(·) and Fine(·) encode the contract's tiered SLA schedule. Overload feeds back into revenue through P_s, which was estimated on the same production data as a sharp threshold:

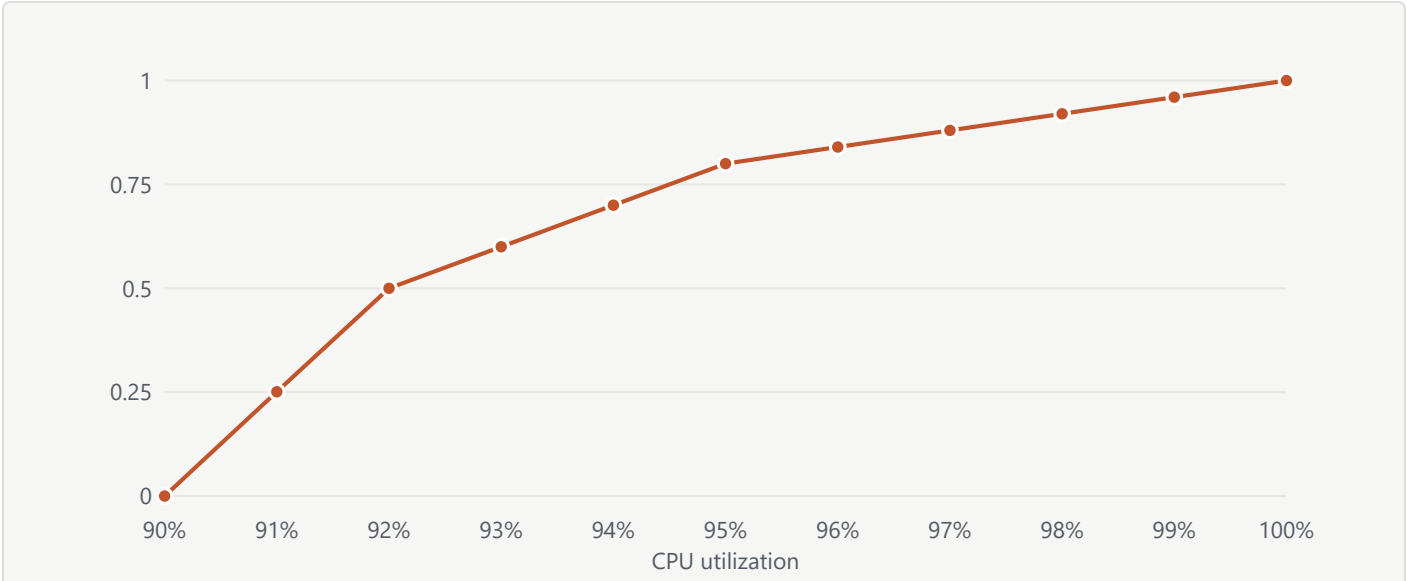


Figure 5. Single-event failure probability $1 - P_s(C)$ as a function of instantaneous CPU utilization. Below ~90% CPU, essentially all events complete within SLA; the curve rises sharply between 90% and 95% (50% failure at 92%, 80%

at 95%), and every event fails once CPU saturates. The narrow transition band is what makes small changes in mean CPU translate into large swings in expected revenue near the operating point.

9 Worked example: annual sizing under contract

To illustrate, we apply the decision-support model to a 20-server fleet handling roughly 386 billion events per year, on a contract paying $\$2 \times 10^{-4}$ per event, with a \$1 M annual fine below 99% availability and a two-tier revenue share above it:

AVAILABILITY TIER	REVENUE SHARE	FINE
< 99%	0%	\$1,000,000
99% – 100%	2%	—
100% (fully met)	4%	—

The event histogram varies from about 0.2 M events per hour off-peak to 4.7 M at the peak hour (19:00). Summing hour-by-hour, the model predicts an expected 366 successful billing dollars per server per day, or approximately \$132 K per server-year and \$2.64 M in expected revenue from the fleet, with residual failure probability concentrated in the two peak hours where CPU brushes 90%.

A quarterly variant of the same model was used to plan hardware growth on a mixed fleet of two server generations. Given about 5% quarterly growth in event volume and 3% growth in operator revenues, the model projects the peak-hour CPU of the older server generation from 57% at baseline to 60% by year-end. Under the same SLA contract, availability remains above 99.9% only if the fleet of the older generation grows from 15 to 18 units over four quarters. The purchase schedule is set by the first quarter in which the predicted CPU crosses the sizing threshold, rather than by a fixed headroom heuristic.

10 Deployment outcomes

The method was applied on two large service-provider deployments. On the first, it drove the hardware sizing analysis for a managed-services engagement and, by replacing a linear extrapolation with the log-normal-based ceiling, supported a reported saving on the order of \$10 M in provisioned hardware over the planning horizon. On the second, it was used internally to compare pre-production soak-test results on a new site with measurements from

an existing production site, giving a quantitative basis for accepting or rejecting the new site's capacity claims.

The method is delivered as an interactive spreadsheet: an operator enters a target transaction rate and a CPU budget, and the tool returns the probability of exceeding that budget and the corresponding expected-revenue impact under the contract's SLA tiers.

11 Scope

Parameters α and β are re-learned for each host and workload from about a month of peak-hour measurements; $\mu_{\log C}$ and $\sigma_{\log C}$ are re-estimated per hour-of-day bin where finer resolution matters. The capacity model describes the regime where CPU is the binding resource; the controlled experiment places the boundary of that regime between 100% and 120% of nominal load for the system studied, beyond which capacity is governed by network or disk I/O and a different model applies. The decision-support model in Sections 8 and 9 assumes an SLA contract whose revenue-share and fine schedule can be expressed as a piecewise function of availability; other contract structures can be handled by substituting RevenueShare(·) and Fine(·) accordingly.